

Proyecto 1: Bitácora de trabajo

Diego Salas Ovares
Frederick Obando Solano

Fundamentos de Arquitectura de Computadores
Instituto Tecnológico de Costa Rica

19 de septiembre de 2025

Registro de participación

[Repositorio en GitHub](#)

Fecha: 2 de septiembre de 2025

Participantes: Frederick

Descripción: Se inicia con el diseño del circuito combinatorio por medio de la creación de las tablas de verdad y mapeos correspondientes. Se plantea el uso de una máquina estado finito de 2 estados (1 bit) que permitan identificar si la puerta ya está abierta o no.

Fecha: 3 de septiembre de 2025

Participantes: Frederick

Descripción: Se crean las tablas de verdad y funciones booleanas del decodificador de un display de 7 segmentos para mostrar los anuncios del circuito combinatorio, decodificador para las entradas del puente H y la máquina de estado finito necesaria para identificar la puerta abierta o cerrada. Queda pendiente proponer las ecuaciones booleanas que representan la combinación de 8 bits producto del registro de corrimiento que permite identificar las contraseñas, ya que es necesario verificar la viabilidad de cada combinación según la frecuencia por utilizar y la funcionalidad del sensor.

Fecha: 5 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se inició la construcción del sistema digital para detección de claves binarias de 8 bits utilizando un DIP switch de 8 vías. Se definió que arriba = 1 y abajo = 0. Se estableció la condición de que la clave debe tener exactamente 3 unos y que el primer bit (*b7*) siempre sea 0.

Se seleccionaron las combinaciones más fáciles de implementar para reducir el uso de compuertas lógicas:

✓ Clave ABRIR: 01110000

✓ Clave CERRAR: 01101000

Esto permite reutilizar señales y minimizar hardware adicional.

Tabla de verdad parcial general:

Clave (b7..b0)	ABRIR	CERRAR	INVÁLIDO
01110000	1	0	0
01101000	0	1	0
Cualquier otra	0	0	1

Fecha: 6 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se diseñó el bloque de inversión usando el **74LS04 (compuerta NOT de 1 entrada)**, generando señales $\overline{b7}$, $\overline{b4}$, $\overline{b2}$, $\overline{b1}$ y $\overline{b0}$. También se construyó el bloque $A = b6 \cdot b5$ con el **74LS08 (compuerta AND de 2 entradas)**.

Se inició el diseño del bloque Z0123 que asegura que los bits bajos $b3, b2, b1, b0$ sean todos 0.

Tabla de verdad del bloque Z0123 (74LS32 OR + 74LS04 NOT):

b3 b2 b1 b0	OR0123	$Z0123 = \overline{(b3 + b2 + b1 + b0)}$
0000	0	1
Otro	1	0

Fecha: 7 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se definió el bloque $Zchain = \overline{b2} \cdot \overline{b1} \cdot \overline{b0}$, implementado con dos compuertas del **74LS08 (AND)** a partir de las salidas invertidas del **74LS04 (NOT)**.

Tabla de verdad del bloque Zchain:

b2 b1 b0	$Zchain = \overline{b2} \cdot \overline{b1} \cdot \overline{b0}$
000	1
Otro	0

Se integraron las funciones finales:

✓ $ABRIR = A \cdot b4 \cdot Z0123 \cdot \overline{b7}$

✓ $CERRAR = A \cdot b3 \cdot \overline{b4} \cdot Zchain \cdot \overline{b7}$

✓ $INVÁLIDO = \overline{(ABRIR + CERRAR)}$

Fecha: 8 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se realizó la prueba con LEDs antes de añadir el display de 7 segmentos. El **74LS08 (AND)** y el **74LS32 (OR)** fueron usados para derivar las salidas ABRIR, CERRAR e INVÁLIDO.

Tabla de verdad salida ABRIR:

Entradas relevantes	$ABRIR = A \cdot b4 \cdot Z0123 \cdot \overline{b7}$
01110000	1
Otro	0

Tabla de verdad salida CERRAR:

Entradas relevantes	$CERRAR = A \cdot b3 \cdot \overline{b4} \cdot Zchain \cdot \overline{b7}$
01101000	1
Otro	0

Tabla de verdad salida INVÁLIDO:

ABRIR, CERRAR	$INVÁLIDO = \overline{(ABRIR + CERRAR)}$
0,0	1
1,0	0
0,1	0

Fecha: 9 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se avanzó al análisis de la etapa de desacople eléctrico. Se determinó que la lógica TTL (5 V) debía estar aislada de la etapa de potencia (motor DC). Se eligió el **L298N (puente H de doble canal)** para el control del motor, debido a su compatibilidad con niveles TTL y capacidad de manejar corrientes mayores.

Fecha: 10 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se diseñó la lógica de control hacia el **L298N (puente H)**:

- ✓ ENA (Motor A) conectado a $ABRIR + CERRAR$ (**74LS32 OR**).
- ✓ IN1 conectado a $ABRIR$ (**74LS08**).
- ✓ IN2 conectado a $CERRAR$ (**74LS08**).
- ✓ INVÁLIDO implica $ENA = 0$ (motor apagado).

Tabla de verdad Motor:

ABRIR	CERRAR	ENA	IN1	IN2	Motor
1	0	1	1	0	Gira A
0	1	1	0	1	Gira B
0	0	0	0	0	Apagado

Fecha: 11 de septiembre de 2025

Participantes: Diego Salas

Descripción: Se integró la etapa de potencia utilizando el **L298N (puente H)**:

- ✓ Motor conectado a OUT1 y OUT2 (canal A del L298N).
- ✓ Fuente de motor (9–12 V) conectada a Vin y GND del módulo.
- ✓ Fuente lógica 5 V conectada a V_lógico y GND común.
- ✓ Jumper regulador retirado para mantener las fuentes desacopladas.
- ✓ Capacitores de 470 μ F y 100 nF añadidos en la entrada de potencia y 100 nF en bornes del motor.

Con esto, el sistema funciona como cerradura electrónica:

- ✓ Clave ABRIR $\rightarrow$ motor abre.
- ✓ Clave CERRAR $\rightarrow$ motor cierra.
- ✓ Cualquier otra combinación $\rightarrow$ motor apagado y LED INVÁLIDO encendido.

Fecha: 12 de septiembre de 2025

Participantes: Frederick Obando

Descripción: Se agrega un bit de estado adicional para representar el estado de lectura en el decodificador de puerta y se actualiza el decodificador de 7 segmentos para interpretar los estados del decodificador de puerta.

- Decodificador de puerta actualizado

$B_{7...0}$	X_3	X_2	X_1	X_0
01110000	1	0	0	0
01101000	0	1	0	0
xxxxxxxx	0	0	1	1

Notese la siguiente nomenclatura: bit de apertura(X_3), bit de cierre(X_2), bit de error(X_1) y bit de lectura(X_0). Además, utilizando suma de productos se obtiene las siguientes ecuaciones booleanas para obtener los bits de apertura y cierre de puerta:

$$\begin{aligned}X_3 &= \overline{B_7}B_6B_5B_4\overline{B_3}\overline{B_2}\overline{B_1}\overline{B_0} \\X_2 &= \overline{B_7}B_6B_5B_4B_3\overline{B_2}\overline{B_1}\overline{B_0}\end{aligned}$$

En el caso de los bits de error y lectura estos se obtienen mediante $\overline{X_3 + X_2}$. Por otro lado para efectos de simplificar la implementación se continúan utilizando bloques de lógica reutilizables como $A = B_6 B_5$, $Z_1 = \overline{B_3 + B_2 + B_1 + B_0}$ (o Z_{CHAIN}), y $Z_2 = \overline{B_2 + B_1 + B_0}$ (o Z_{123}) tal que:

$$\begin{aligned} X_3 &= \overline{B_7} \cdot A \cdot B_4 \cdot Z_1 \\ X_2 &= \overline{B_7} \cdot A \cdot \overline{B_4} B_3 \cdot Z_2 \end{aligned}$$

- Decodificador de puente H

X_3	X_2	ENA	IN1	IN2
1	0	1	1	0
0	1	1	0	1
0	0	0	0	0

Las ecuaciones booleanas para obtener los bits de salida son:

$$\begin{aligned} ENA &= X_3 \oplus X_2 \\ IN1 &= X_3 \overline{X_2} \\ IN2 &= \overline{X_3} X_2 \end{aligned}$$

- Decodificador de display de 7 segmentos

X_3	X_2	X_1	$L_{6...0}$	Símbolo
1	0	0	1110111	A
0	1	0	0111001	C
0	0	1	1111001	E

Las ecuaciones booleanas para las salidas se obtienen por medio de suma de productos tal que:

$$\begin{aligned} L_6 &= X_3 \overline{X_2} \overline{X_1} + \overline{X_3} \overline{X_2} X_1 \\ L_5 &= 1 \\ L_4 &= 1 \\ L_3 &= \overline{X_3} X_2 \overline{X_1} + \overline{X_3} \overline{X_2} X_1 \\ L_2 &= X_3 \overline{X_2} \overline{X_1} \\ L_1 &= X_3 \overline{X_2} X_1 \\ L_0 &= 1 \end{aligned}$$

Dado que hay operaciones que se repiten se pueden generar los bloques $f_1 = X_3 \overline{X_2} \overline{X_1}$, $f_2 = \overline{X_3} \overline{X_2} X_1$ y $f_3 = \overline{X_3} X_2 \overline{X_1}$ tal que ciertos bloques reducen a:

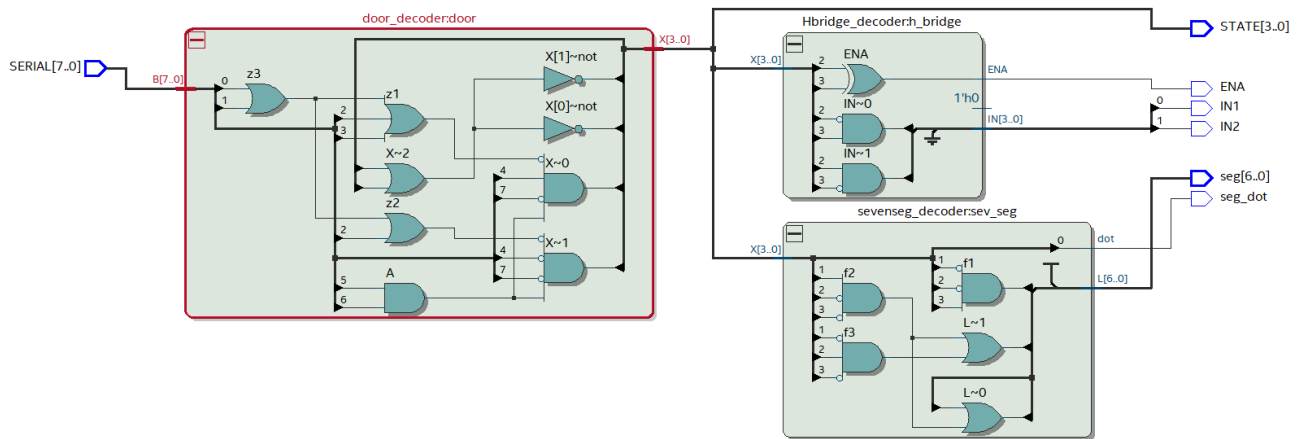
$$\begin{aligned}
L_6 &= f_1 + f_2 \\
L_3 &= f_3 + f_2 \\
L_2 &= f_1 \\
L_1 &= f_1
\end{aligned}$$

Fecha: 13 de septiembre de 2025

Participantes: Frederick Obando

Descripción: Se diseña el esquemático del circuito combinatorio que contemple los tres módulos principales: decodificador de puerta, decodificador para el puente H y el decodificador para el display de 7 segmentos (ver Figura 1). Este se diseño utilizando el Quartus Prime Lite.

Figura 1. Esquemático del circuito combinatorio



Para la implementacion se hace un cambio de compuertas TTL a CMOS en todos los módulos para evitar afectar el comportamiento del display de 7 segmentos. El listado completo se muestra a continuación:

- ✓ Compuerta AND: CD4081
- ✓ Compuerta OR: CD4071
- ✓ Compuerta NOR: CD4001
- ✓ Compuerta NOT: CD4069
- ✓ Compuerta XOR: CD4070

Fecha: 17 de septiembre de 2025

Participantes: Frederick Obando

Descripción: Se implementa el modulo decodificador para display de 7 segmentos según lo establecido en el esquemático de la [Figura 1](#) logrando así:

- ✓ Identificar el abrir de la puerta al recibir la contraseña establecida con una 'A' en el display de 7 segmentos.
- ✓ Identificar el cierre de la puerta al recibir la segunda contraseña establecida con una 'C' en el display.
- ✓ Identificar cuando la contraseña ingresada es incorrecta para ambos casos con una letra 'E' en el display.
- ✓ Indicar que el sistema esta haciendo una lectura indicado con el punto decimal del display de 7 segmentos, si alguna de las contraseñas coincide este punto se apaga,

Es importante notar que para la implementación completa de los módulos se tuvieron que agregar buffers (integrado CD4050) en las salidas del decodificador de puerta y el decodificador de 7 segmentos para ajustar los niveles lógicos y que los bloques lógicos operaran correctamente.

Fecha: 17 de setiembre de 2025

Participantes: Diego Salas

Descripción: Se documenta la configuración del **sensor SW-420** y la **generación del reloj** para el 74HC164 con el fin de capturar eventos de vibración en ventanas temporales fijas y mostrarlos sin corrimiento visible.

Configuración del sensor SW-420 (DO activo en bajo, reposo en alto):

- ✓ DO del SW-420 conectado a PIN_SENSOR = D12, VCC a 5 V y GND común.
- ✓ Entrada configurada como INPUT_PULLUP: en reposo lee HIGH; con vibración cae a LOW.
- ✓ *Criterio de ventana (hipersensible):* si en cualquier instante de la ventana se detecta LOW, se marca la ventana con 1; de lo contrario 0.
- ✓ Recomendaciones de estabilidad: ajustar el potenciómetro del módulo para reducir falsos positivos y colocar un capacitor de 100 nF entre VCC y GND del módulo (desacoplo).

Parámetro	Valor	Observación
WINDOW_MS	2000 ms	Duración de ventana de medición (LED13 encendido).
GAP_MS	1000 ms	Separación entre ventanas (LED13 apagado).
Fases	WINDOW_ACTIVE / GAP_SEP / STOPPED	Flujo controlado por <code>millis()</code> .

Generación de reloj para 74HC164 (Arduino D3 → CLK):

- ✓ PIN_CLK = D3 inicia en LOW; el dato en PIN_DATA_A = D2 se establece antes del flanco.
- ✓ Pulso de reloj mediante `clkHigh()/clkLow()` con retardos de $\approx 2 \mu\text{s}$ asegura *setup/hold* holgados para la familia HC.
- ✓ Línea B del 74HC164 fijada a 5 V (AND interno con A); /CLR a 5 V para evitar borrado asíncrono.
- ✓ Desacoplo de 100 nF entre pines 14 (VCC) y 7 (GND) del 74HC164, colocado lo más cerca posible del encapsulado.

Indicadores:

- ✓ LED13 encendido $\Rightarrow$ ventana activa; apagado $\Rightarrow$ separación.
- ✓ Al cerrar cada ventana, se actualiza la “matriz” de LEDs escribiendo el frame completo al 74HC164.

Fecha: 18 de setiembre de 2025

Participantes: Diego Salas

Descripción: Se define el **mapeo entre bits de ventana y salidas del 74HC164** y el **cableado de LEDs en modo source**, además de una rutina de verificación para validar el orden visual.

Cableado del 74HC164 y LEDs (modo source):

- ✓ VCC (pin 14) a 5 V; GND (pin 7) a GND; CLK (pin 8) a D3; A (pin 1) a D2; B (pin 2) a 5 V; /CLR (pin 9) a 5 V.
- ✓ Cada salida $Q_x \rightarrow$ resistencia (330Ω – $1 \text{ k}\Omega$) $\rightarrow$ **ánodo** del LED; **cátodo** del LED a GND.
- ✓ Escritura *sin inversión*: $\text{bit}=1 \Rightarrow Q_x=1 \Rightarrow$ LED encendido.

Mapa bits $\leftrightarrow$ 74HC164 $\leftrightarrow$ LEDs:

Salida	Pin 74HC164	bits[i]	LED físico
QA	3	bits[0]	LED0
QB	4	bits[1]	LED1
QC	5	bits[2]	LED2
QD	6	bits[3]	LED3
QE	10	bits[4]	LED4
QF	11	bits[5]	LED5
QG	12	bits[6]	LED6
QH	13	bits[7]	LED7

Correspondencia lógica y orden de envío:

- ✓ QA siempre refleja el **último** bit desplazado; por eso se envían los datos en el orden `bits[7] → ... → bits[0]` para que `bits[0]` termine en QA.
- ✓ Con el mapa anterior, la ventana 0 se ve en LED0 (QA), y la ventana 7 en LED7 (QH).